

UNIFECAP

Análise e Desenvolvimento de Sistemas

RELATÓRIO TEÓRICO

RentSmart - Sistema de Orçamento de Locação

Algorithmic Thinking & Introduction to Object-Oriented Programming

Aluno:Otávio Augusto Leite dos Santos

1. Introdução

Este relatório apresenta a fundamentação teórica e o desenvolvimento do **RentSmart**, um programa em Python que gera orçamentos de locação de imóveis. O sistema trabalha com três tipos de imóvel (apartamento, casa e estúdio), calcula o aluguel mensal conforme regras de negócio específicas de cada tipo, parcela uma taxa contratual e produz uma projeção financeira de 12 meses em arquivo CSV.

O **objetivo** do trabalho é aplicar, em um problema realista, os conceitos vistos na disciplina: pensamento algorítmico (decomposição, abstração e construção de algoritmos) e programação orientada a objetos (classes, herança e polimorfismo).

O relatório está organizado da seguinte forma: a Seção 2 revisa os conceitos teóricos; a Seção 3 analisa o problema; a Seção 4 mostra a modelagem; a Seção 5 descreve a implementação; a Seção 6 apresenta os testes; a Seção 7 discute limitações e melhorias; e a Seção 8 traz a conclusão.

2. Fundamentação Teórica

2.1 Pensamento algorítmico

Pensamento algorítmico é a capacidade de resolver problemas por meio de uma sequência finita e ordenada de passos. Seus pilares são:

- **Decomposição:** dividir o problema em partes menores. No RentSmart: escolher o imóvel, calcular o aluguel, parcelar a taxa e gerar o CSV.
- **Reconhecimento de padrões:** os três imóveis compartilham atributos (tipo, aluguel base, quartos) e comportamento (calcular aluguel).
- **Abstração:** a classe `Imovel` guarda apenas o que é comum, ocultando detalhes de cada tipo.
- **Algoritmo:** a sequência de passos do programa, representada na Seção 4 por fluxograma.

2.2 Estruturas de controle

O programa usa **sequência** (instruções executadas em ordem), **seleção** (`if/elif/else`, para escolher o tipo de imóvel e aplicar adicionais) e **repetição** (`for`, para gerar os 12 meses do CSV).

2.3 Programação orientada a objetos

A POO organiza o software em **objetos**, instâncias de **classes**. Uma classe define **atributos** (dados) e **métodos** (comportamentos). O **construtor** (`__init__`) inicializa o objeto no momento da criação.

2.4 Os pilares da POO

Pilar	Conceito	No RentSmart
Herança	Uma classe reaproveita atributos e métodos de outra.	Apartamento, Casa e Estudio herdam de Imovel via <code>super().__init__()</code> .
Polimorfismo	Mesmo método, comportamentos diferentes conforme a classe.	<code>calcularAluguel()</code> é sobrescrito em cada filha; <code>home()</code> o chama sem saber o tipo.
Abstração	Modelar apenas o que é essencial.	Imovel é um modelo genérico, com <code>calcularAluguel()</code> vazio (pass).
Encapsulamento	Proteger o estado interno do objeto.	Atributos são públicos; ver melhoria na Seção 7.

2.5 Manipulação de arquivos CSV

CSV (*Comma-Separated Values*) é um formato de texto tabular, aberto por planilhas como Excel. O módulo `csv` do Python fornece o `writer`, e o bloco `with open(...)` garante o fechamento do arquivo. Foram usados `newline=''` (evita linhas em branco) e `encoding='utf-8'` (preserva acentos).

3. Análise do Problema

3.1 Entradas e saídas

- **Entradas:** tipo de imóvel; quantidade de quartos (apartamento e casa); garagem; presença de crianças (apartamento); vagas (estúdio); número de parcelas da taxa.
- **Saídas:** resumo do orçamento no terminal e arquivo CSV com a projeção de 12 meses.

3.2 Regras de negócio

Imóvel	Aluguel base	Adicionais e descontos
Apartamento	R\$ 700,00	+ R\$ 200 (2 quartos); + R\$ 300 (garagem); - 5% se não houver crianças (aplicado por último, sobre o total)
Casa	R\$ 900,00	+ R\$ 250 (2 quartos); + R\$ 300 (garagem)
Estúdio	R\$ 1.200,00 (1 quarto)	+ R\$ 250 (até 2 vagas); acima de 2 vagas, + R\$ 250 mais R\$ 60 por vaga extra

Taxa contratual: valor fixo de R\$ 2.000,00, parcelável em 1 a 5 vezes sem juros. Valores fora desse intervalo são convertidos automaticamente em 1 parcela. No CSV, a parcela é cobrada nos primeiros *N* meses e vale R\$ 0,00 nos demais.

4. Modelagem e Projeto da Solução

4.1 Diagrama de classes

A figura abaixo mostra a hierarquia de classes. `Imovel` é a superclasse; as três subclasses herdam seus atributos e sobrescrevem `calcularAluguel()`.

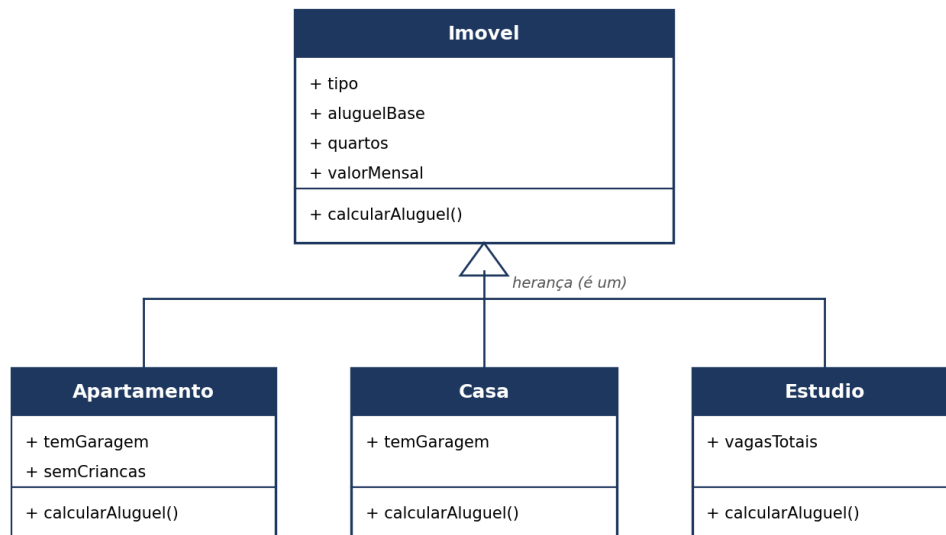


Figura 1 - Diagrama de classes do RentSmart.

4.2 Fluxograma do programa

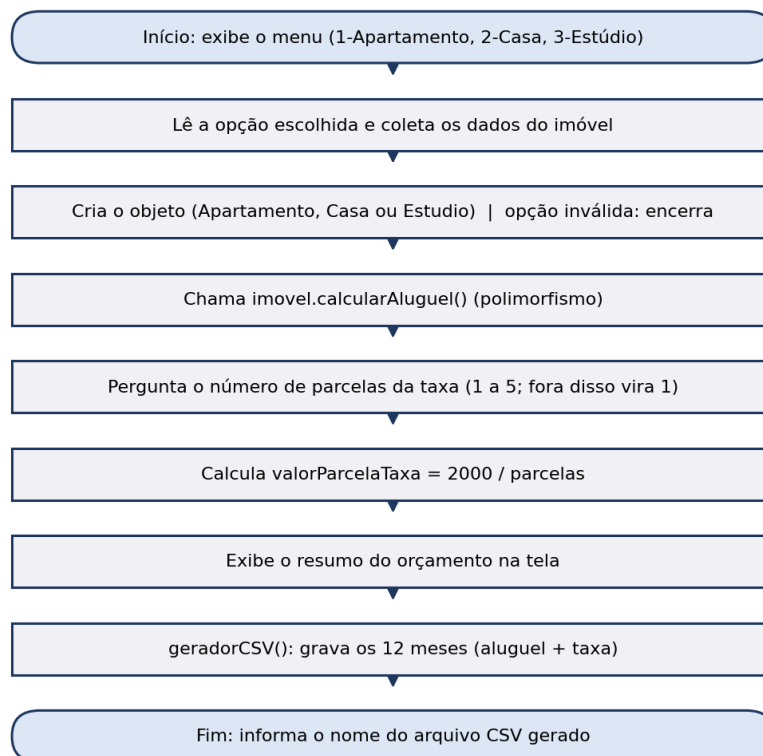


Figura 2 - Fluxo de execução da função home().

4.3 Pseudocódigo do cálculo (apartamento)

```

valor <- 700
se quartos = 2 entao valor <- valor + 200
se temGaragem entao valor <- valor + 300
se semCrianças entao valor <- valor * 0,95
retorne valor
  
```

5. Implementação

5.1 Estrutura do código

O programa está em um único arquivo, `main.py`, com três partes: (1) as classes do domínio; (2) a função `geradorCSV()`; e (3) a função `home()`, que concentra o menu e a interação com o usuário. O trecho final `if __name__ == '__main__':` garante que `home()` só rode quando o arquivo é executado diretamente, e não quando importado.

5.2 Herança e construtor

Cada subclasse chama `super().__init__()` informando tipo, aluguel base e quartos, reaproveitando o construtor da superclasse:

```
class Casa(Imovel):
    def __init__(self, quartos, temGaragem):
        super().__init__("Casa", 900.00, quartos)
        self.temGaragem = temGaragem
```

5.3 Polimorfismo

Em `home()`, a variável `imovel` recebe um objeto de uma das três classes e a mesma linha de código calcula o valor correto em cada caso:

```
aluguelFinal = imovel.calcularAluguel()
```

5.4 Geração do CSV

A função `geradorCSV()` percorre os meses de 1 a 12 com `for`. Uma expressão condicional define a taxa de cada mês, e o total mensal soma aluguel e taxa:

```
taxaMes = valorParcelasTaxa if mes <= parcelasTaxa else 0.0
totalMes = aluguelMensal + taxaMes
```

O arquivo gerado se chama *Projeto Locação 12 meses.csv* e tem as colunas Mes, Aluguel Mensal (R\$), Taxa Contratual (R\$) e Total no Mês (R\$).

6. Testes e Resultados

Os casos abaixo foram calculados manualmente a partir das regras da Seção 3. **Confirme os resultados executando o programa** e complete a coluna "Obtido".

#	Entrada	Esperado	Obtido
1	Apartamento, 2 quartos, com garagem, sem crianças	$(700+200+300) \times 0,95 = \text{R\$ } 1.140,00$	
2	Apartamento, 1 quarto, sem garagem, com crianças	R\$ 700,00	
3	Casa, 2 quartos, com garagem	$900+250+300 = \text{R\$ } 1.450,00$	
4	Casa, 1 quarto, sem garagem	R\$ 900,00	
5	Estúdio, 2 vagas	$1.200+250 = \text{R\$ } 1.450,00$	
6	Estúdio, 4 vagas	$1.200+250+(2 \times 60) = \text{R\$ } 1.570,00$	

#	Entrada	Esperado	Obtido
7	Taxa em 4 parcelas	R\$ 500,00 nos meses 1 a 4; R\$ 0,00 nos demais	
8	Parcelas = 8 (inválido)	Assume 1 parcela de R\$ 2.000,00	
9	Opção de menu = 7	Exibe "Opção invalida!!!" e encerra	

Exemplo de saída esperada (caso 1 com taxa em 4 parcelas): mês 1 a 4 com total de R\$ 1.640,00 (1.140,00 + 500,00) e mês 5 a 12 com total de R\$ 1.140,00.

[INSIRA AQUI: print do terminal e print do CSV aberto no Excel]

7. Limitações e Melhorias

A análise crítica do código identificou os pontos abaixo, que seriam tratados em uma versão futura:

- **Validação de entrada:** `int(input())` interrompe o programa se o usuário digitar texto. Um bloco `try/except ValueError` resolveria.
- **Mínimo de vagas no estúdio:** o menu informa "mínimo 2", mas o código não impede valores menores.
- **Estado acumulado:** `calcularAluguel()` altera `self.valorMensal`; chamá-lo duas vezes somaria os adicionais novamente. Usar uma variável local evita o problema.
- **Classe abstrata:** `Imovel.calcularAluguel()` poderia ser abstrato (módulo `abc`), obrigando as filhas a implementá-lo.
- **Encapsulamento:** atributos privados com `@property` protegeriam os dados.
- **Parcelas inválidas:** o programa corrige em silêncio para 1 parcela; o ideal seria avisar o usuário ou pedir novamente.
- **Interface:** separar a lógica de negócio da interação (menu) facilitaria testes automatizados.

8. Conclusão

O RentSmart atendeu ao objetivo proposto: calcular orçamentos de locação de três tipos de imóvel e gerar a projeção financeira de 12 meses em CSV. A decomposição do problema permitiu dividir o sistema em classes e funções pequenas, e a herança evitou repetir atributos comuns.

O trabalho mostrou na prática como o polimorfismo simplifica o código principal, que não precisa saber o tipo do imóvel para calcular o aluguel. Também evidenciou a importância da validação de dados e do encapsulamento, apontados como melhorias.

Apêndice - Código-fonte (main.py)

```
import csv as cv #importa o gerador de arquivos CSV mudando o nome pra cv

#classe principal dos imoveis
#ela vai servir como base pro apartamento, casa e estudio
class Imovel:

    def __init__(self, tipo, aluguelBase, quartos):
        self.tipo = tipo
        self.aluguelBase = aluguelBase
        self.quartos = quartos
        self.valorMensal = aluguelBase #começa o aluguel com o valor base

    def calcularAluguel(self):
        #esse metodo vai ser sobrescrito pelas classes filhas
        pass

#classe apartamento herdando as informações da classe Imovel
class Apartamento(Imovel):

    def __init__(self, quartos, temGaragem, semCrianças):

        # hama o __init__ da classe Imovel
        #define o tipo como apartamento e o aluguel base como R$ 700
        super().__init__("Apartamento", 700.00, quartos)

        self.temGaragem = temGaragem
        self.semCrianças = semCrianças

    def calcularAluguel(self):

        #se o apartamento tiver 2 quartos adiciona R$ 200 no aluguel
        if self.quartos == 2:
            self.valorMensal += 200.00

        #se tiver garagem adiciona R$ 300 no aluguel
        if self.temGaragem:
            self.valorMensal += 300.00

        #se não tiver crianças recebe 5% de desconto
        if self.semCrianças:
            self.valorMensal *= 0.95

        #retorna o valor final do aluguel
        return self.valorMensal

#classe casa herdando as informações da classe Imovel
class Casa(Imovel):

    def __init__(self, quartos, temGaragem):

        #chama o __init__ da classe Imovel
        #define o tipo como casa e o aluguel base como R$ 900
        super().__init__("Casa", 900.00, quartos)

        self.temGaragem = temGaragem

    def calcularAluguel(self):

        #se a casa tiver 2 quartos adiciona R$ 250
        if self.quartos == 2:
            self.valorMensal += 250.00

        #se tiver garagem adiciona R$ 300
        if self.temGaragem:
            self.valorMensal += 300.00

        #retorna o valor final do aluguel
```

```

        return self.valorMensal

#classe estudio herdando as informações da classe Imovel
class Estudio(Imovel):

    def __init__(self, vagasTotais):

        #chama o __init__ da classe Imovel
        #define o tipo como estudio, aluguel base de R$ 1.200 e 1 quarto
        super().__init__("Estudio", 1200.00, 1)

        self.vagasTotais = vagasTotais

    def calcularAluguel(self):

        #se tiver até 2 vagas adiciona R$ 250
        if self.vagasTotais <= 2:
            self.valorMensal += 250.00

        else:
            #calcula quantas vagas passaram das 2 vagas permitidas
            vagasExtras = self.vagasTotais - 2

            #adiciona R$ 250 pelas vagas iniciais
            #e mais R$ 60 para cada vaga extra
            self.valorMensal += 250.00 + (vagasExtras * 60.00)

        #retorna o valor final do aluguel
        return self.valorMensal

#função responsável por criar o arquivo CSV
def geradorCSV(nomeArqui, aluguelMensal, parcelasTaxa, valorParcelasTaxa):

    #abre/cria o arquivo CSV
    #'w' significa que vai escrever no arquivo
    #newline evita linhas vazias no CSV
    #encoding utf-8 permite caracteres como ç e acentos
    with open(nomeArqui, mode='w', newline='', encoding='utf-8') as arquivo:

        #cria o escritor do arquivo CSV
        escritor = cv.writer(arquivo)

        #cria o cabeçalho da tabela
        escritor.writerow([
            'Mes',
            'Aluguel Mensal (R$)',
            'Taxa Contratual (R$)',
            'Total no Mês (R$)'
        ])

        #vai gerar os dados dos 12 meses
        for mes in range(1, 13):

            #enquanto estiver dentro da quantidade de parcelas
            #adiciona o valor da parcela da taxa contratual
            #depois que terminar as parcelas, fica R$ 0
            taxaMes = valorParcelasTaxa if mes <= parcelasTaxa else 0.0

            # soma aluguel + taxa contratual daquele mês
            totalMes = aluguelMensal + taxaMes

            #escreve os dados do mês no arquivo CSV
            escritor.writerow([
                mes,
                f'{aluguelMensal:.2f}',
                f'{taxaMes:.2f}',
                f'{totalMes:.2f}'
            ])

#função principal do programa

```

```

#aqui vai ficar o menu e toda interação com o usuario
def home():

    #menu do aplicativo
    print('=== RentSmart - Sistema de Orçamento de Locação ===')
    print('Escolha o tipo de imóvel:')
    print('1- Apartamento (Base: R$ 700,00)')
    print('2- Casa (Base: R$ 900,00)')
    print('3- Estúdio (Base: R$ 1.200,00)')

    #pega a opção escolhida pelo cliente
    opc = int(input('Escolha uma das opções: '))

    #começa a variavel vazia
    #depois ela vai receber Apartamento, Casa ou Estudio
    imovel = None

    #escolhas do cliente
    if opc == 1:

        #pergunta a quantidade de quartos
        quartos = int(input('Quantidade de quartos 1 ou 2: '))

        #pergunta se possui garagem
        #se o usuario digitar s, garagem recebe True
        #se digitar qualquer outra coisa recebe False
        garagem = input(
            'Possui vaga de garagem? (s/n): '
        ).lower() == 's'

        #pergunta se possui crianças
        #se responder n, semCrianca recebe True
        semCrianca = input(
            'Possui crianças? (s/n): '
        ).lower() == 'n'

        #cria um objeto da classe Apartamento
        imovel = Apartamento(quartos, garagem, semCrianca)

    elif opc == 2:

        #pergunta a quantidade de quartos
        quartos = int(input('Quantidade de quartos 1 ou 2: '))

        #pergunta se possui garagem
        garagem = input(
            'Possui vaga de garagem? (s/n): '
        ).lower() == 's'

        #cria um objeto da classe Casa
        imovel = Casa(quartos, garagem)

    elif opc == 3:

        #pergunta quantas vagas o cliente deseja
        vagas = int(
            input('Quantas vagas de estacionamento deseja (minimo 2)? ')
        )

        #cria um objeto da classe Estudio
        imovel = Estudio(vagas)

    else:

        #aso o cliente escolha uma opção errada
        #retorna uma mensagem e encerra a função
        print('Opção invalida!!!')
        return

    #chama o metodo calcularAluguel()
    #cada classe possui sua propria forma de calcular
    aluguelFinal = imovel.calcularAluguel()

```

```

#define o valor total da taxa contratual
taxaContratual = 2000.00

#pergunta em quantas vezes o cliente deseja parcelar a taxa
parcelas = int(
    input(
        'Em quantas vezes deseja parcelar a taxa contratual '
        'de R$2.000 (1 a 5): '
    )
)

#verifica se a quantidade de parcelas está dentro do permitido
if parcelas < 1 or parcelas > 5:

    #caso seja uma quantidade invalida,
    #automaticamente define como 1 parcela
    parcelas = 1

#calcula o valor de cada parcela da taxa contratual
valorParcelaTaxa = taxaContratual / parcelas

#mostra o resumo do orçamento na tela
print("\n--- RESUMO DO ORÇAMENTO ---")
print(f"Tipo de Imóvel: {imovel.tipo}")
print(f"Valor Mensal do Aluguel: R$ {aluguelFinal:.2f}")

print(
    f"Taxa Contratual Total: R$ {taxaContratual:.2f} "
    f"(dividida em {parcelas}x de R$ {valorParcelaTaxa:.2f})"
)

#define o nome do arquivo CSV que será criado
nomeCSV = 'Projeto Locação 12 meses.csv'

#chama a função responsável por gerar o arquivo CSV
geradorCSV(
    nomeCSV,
    aluguelFinal,
    parcelas,
    valorParcelaTaxa
)

#informa para o usuario que o arquivo foi criado
print(
    f'\nProjeção de 12 meses gerada com sucesso '
    f'no arquivo: {nomeCSV}'
)

#verifica se esse arquivo está sendo executado diretamente
#se estiver, chama a função principal home()
if __name__ == '__main__':
    home()

```